

Partitions satisfaisantes dans les graphes

Travail d'Étude et de Recherche — Master 1 Algorithmique

Ivan LEJEUNE

Encadrant : Stéphane BESSY

Université de Montpellier
Faculté des Sciences de Montpellier

25 mai 2026

- 1 Introduction et définitions
- 2 Motivations et état de l'art
- 3 Résultats pour les graphes k -réguliers ($k \leq 4$)
- 4 Le saut de complexité : $k \geq 5$
- 5 Méthodes algorithmiques
- 6 Conclusion et perspectives

Cadre général

Soit $G = (V, E)$ un graphe simple, non orienté, fini.

On étudie l'existence d'une **partition satisfaisante** de ses sommets : une notion qui formalise la cohésion interne au sein de sous-structures de graphes.

Definition (Degrés interne et externe)

Soit (A, B) une partition de V et $x \in V$.

- Si $x \in A$: $d_{\text{int}}(x) = |N(x) \cap A|$, $d_{\text{ext}}(x) = |N(x) \cap B|$
- Si $x \in B$: $d_{\text{int}}(x) = |N(x) \cap B|$, $d_{\text{ext}}(x) = |N(x) \cap A|$

Partition satisfaisante

Definition (Partition satisfaisante)

Une partition (A, B) de V est **satisfaisante** si pour tout $x \in V$:

$$d_{\text{int}}(x) \geq d_{\text{ext}}(x)$$

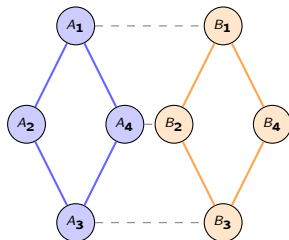
Un sommet vérifiant cette condition est dit *satisfait*.

Le problème de décision associé est noté SATISFACTORY GRAPH PARTITION.

Intuition

Chaque sommet doit avoir *au moins autant* de voisins dans sa propre communauté que dans l'autre.

⇒ notion de **cohésion** ou **communauté**.



Réseaux sociaux & biologiques

Sommets = individus ou gènes ;
arêtes = interactions.

Une partition satisfaisante isole des **communautés stables** où l'activité interne prime.

Théorie des jeux

Modélise la **formation d'alliances défensives** : un groupe A est stable si chaque membre trouve plus de soutien en interne qu'il n'affronte de menaces externes.

Intelligence artificielle

Pilier du *clustering* non supervisé.

Densité interne $>$ connectivité externe $\Rightarrow$ meilleure **robustesse** pour la classification.

Références principales

- Bazgan, Tuza, Vanderpooten (2010) — travaux fondateurs
- Yero & Rodríguez-Velázquez (2018) — alliances défensives
- Gaikwad, Maity, Tripathi (2020) — algorithmes structurels
- Bärnkopf, Nagy, Paulovics (2024) — cas 5-régulier et au-delà

Résultats de complexité connus

- **NP-complet** en général
- **Polynomial** pour :
 - graphes à clique-width bornée
 - $\Delta(G) \leq 4$

Notre approche

- Construire des **contre-exemples** à la conjecture
- Identifier des **invariants structurels** minimaux bloquants
- Expérimentation algorithmique sur les petits graphes

Conjecture (ouverte pour $k \geq 5$)

Pour tout entier k , tous les graphes k -réguliers, **sauf un nombre fini d'exceptions**, admettent une partition satisfaisante.

Réduction aux graphes connexes

Proposition. Tout graphe *non connexe* admet une partition satisfaisante.

Preuve. Prendre A = une composante connexe, $B = V \setminus A$. Alors $d_{\text{ext}}(x) = 0$ pour tout x , donc la partition est satisfaisante. □

⇒ Il suffit d'étudier les graphes k -réguliers **connexes**.

Cas $k = 1$ et $k = 2$

$k = 1$: graphe K_2

L'unique graphe 1-régulier connexe est K_2 .

Séparer les deux sommets donne $d_{\text{int}} = 0$,
 $d_{\text{ext}} = 1$ pour chacun.

K_2 n'admet *pas* de partition satisfaisante.

$k = 2$: cycle C_n

- $n = 3$: $G \simeq K_3$, **pas de partition**.
- $n \geq 4$: la partition $(\{x, y\}, V \setminus \{x, y\})$ avec $(x, y) \in E$ est satisfaisante.
 - Pour $z \in \{x, y\}$: $d_{\text{int}} = d_{\text{ext}} = 1$ ✓
 - Pour $z \notin \{x, y\}$: $d_{\text{int}} \geq 1$, $d_{\text{ext}} \leq 1$ ✓

Bilan $k \leq 2$

Exceptions finies : K_2 et K_3 . La conjecture est vérifiée.

Cas $k = 3$ — analyse par le plus petit cycle

Stratégie

Soit C le plus petit cycle de G (graphe 3-régulier connexe).

Si $|C| = 3$ (triangle) :

- Voisin relié aux 3 sommets $\Rightarrow G \simeq K_4$
(exception)
- Voisin relié à 2 sommets $\Rightarrow$ élargir
 $A = C \cup \{x, \dots\}$ donne une partition satisfaisante
- Voisins reliés à ≤ 1 sommet $\Rightarrow (C, V \setminus C)$ satisfaisante

Si $|C| \geq 4$:

- Voisin relié à 3 sommets $\Rightarrow$ cycle plus court, contradiction
- Voisin relié à ≤ 1 sommet $\Rightarrow (C, V \setminus C)$ satisfaisante
- Voisin relié à 2 sommets opposés : soit $G \simeq K_{3,3}$ (exception), soit partition satisfaisante en ajoutant ces sommets

Bilan $k = 3$

Deux exceptions : K_4 et $K_{3,3}$. Tous les autres graphes 3-réguliers connexes admettent une partition satisfaisante.

Cas $k = 4$ — lemme clé

Lemma

Tout graphe 4-régulier connexe, sauf K_5 , contient **au moins deux cycles sommet-disjoints**.

Esquisse de preuve.

Soit C le cycle minimal ($|C| = c$). On suppose par l'absurde que $T = G \setminus C$ est une forêt ($t = |T|$).

- 1 Lemme des poignées de main dans T : $\sum_{u \in T} d_{\text{int}}(u) \leq 2t - 2$
- 2 Comptage des arêtes $C \leftrightarrow T$ (exactement $2c$) : $\sum_{u \in T} d_{\text{int}}(u) = 4t - 2c$
- 3 Combinaison : $c \geq t + 1$
- 4 Toute feuille $u \in T$ a ≥ 3 voisins sur C , créant un cycle de longueur $\leq \lfloor c/3 \rfloor + 2 < c$ pour $c \geq 4$.
- 5 La seule possibilité : $c = 3$, $t \leq 2$, $|V| \leq 5 \Rightarrow G \simeq K_5$. **Contradiction.**



Cas $k = 4$ — théorème d'existence

Theorem

Tout graphe 4-régulier connexe, sauf K_5 , admet une partition satisfaisante.

Construction de la partition.

Soient C_1, C_2 deux cycles sommet-disjoints (lemme précédent).

- ❶ Poser $A = \{v \in V : v \in C_1 \text{ ou } |N(v) \cap C_1| \geq 2\}$, $B = V \setminus A$.
- ❷ Pour tout $x \in A$: $d_{\text{int}}(x) \geq 2$ et $d_{\text{ext}}(x) \leq 2$ (4-régularité). ✓
- ❸ Tant qu'un sommet de B est non satisfait : le déplacer vers A .
 - Chaque déplacement réduit strictement la coupe $\Rightarrow$ terminaison. ✓
 - $C_2 \subset B$ reste intact : B n'est jamais vidé. ✓



Bilan $k = 4$

Une seule exception : K_5 . La conjecture est vérifiée.

Notion de t -noyau

Definition (t -noyau)

Un t -noyau de G est un ensemble $N \subseteq V$ tel que $\deg_N(u) \geq t$ pour tout $u \in N$.

Lien avec les partitions satisfaisantes

Proposition. Si G est k -régulier et contient deux $\lceil k/2 \rceil$ -noyaux disjoints N_1, N_2 , alors G admet une partition satisfaisante.

Idée : partir de $(N_1, V \setminus N_1)$ et déplacer itérativement les sommets non satisfaits. Le nombre de coupures décroît ; N_1 et N_2 garantissent que les deux parties restent non vides.

$k = 4 : t = 2$

Un 2-noyau = un **cycle**. Facile à trouver !

$k = 5 : t = 3$

Un 3-noyau = sous-graphe de degré ≥ 3 .
Beaucoup plus contraint et difficile à caractériser.

Condition suffisante pour deux noyaux disjoints

Proposition

Soit G un graphe k -régulier à n sommets, $t = \lceil k/2 \rceil$.

Si G contient un t -noyau de taille $p < \frac{n}{\lceil k/2 \rceil + 1}$, alors G contient **au moins deux** t -noyaux disjoints.

Idée.

Un noyau de taille p a $p \cdot \lfloor k/2 \rfloor$ arêtes sortantes. Ajouter un sommet non satisfait réduit ce nombre d'au moins 1. Si $p \cdot (\lceil k/2 \rceil + 1) < n$, le processus converge avant de vider $V \setminus N$, produisant un second noyau dans la partie restante. □

Conséquence pour la conjecture

Si l'on peut borner la taille minimale d'un premier t -noyau dans les graphes k -réguliers à *partir d'une certaine taille*, alors il n'existe qu'un **nombre fini de contre-exemples** pour ce k .

Le cas $k = 6$ et les cas ouverts

Résultat connu pour $k = 6$

La conjecture est **vérifiée** pour tous les graphes 6-réguliers à **au moins 12 sommets** (Bärnkopf, Nagy, Paulovics — 2024).

Idée de preuve : comptage fin des arêtes traversant la partition $\Rightarrow$ existence de noyaux disjoints.

Cas $k = 5$ et $k \geq 7$

- Toujours **ouverts** (mai 2026)
- Aucun contre-exemple trouvé pour les petits graphes
- Complexité croissante des t -noyaux nécessaires

Récapitulatif

k	Exceptions	Statut
1	K_2	Prouvé
2	K_3	Prouvé
3	$K_4, K_{3,3}$	Prouvé
4	K_5	Prouvé
5	?	Ouvert
6	≤ 11 sommets	Prouvé
≥ 7	?	Ouvert

Trois approches implémentées en Python

- ➊ **Heuristique aléatoire** : partitions initiales aléatoires + rééquilibrage glouton
- ➋ **Algorithme basé sur les noyaux** : part d'un cycle (2-noyau), étend itérativement
- ➌ **Recherche exhaustive** : teste toutes les 2^{n-1} partitions (petits graphes)

Complexités

Brute-force	$O(2^n)$
Glouton	$O(k^2 n)$
Basé noyaux	$O(k^2 n)$ avec garantie

Principe du glouton

Partant de (A, B) , déplacer un sommet non satisfait vers l'autre partie.

Chaque déplacement réduit $|\text{coupe}|$ d'au moins 1.
Convergence en $O(|E|)$ étapes.

Algorithme glouton (extrait)

```
def satisfying_partition(G, max_attempts=100):
    nodes = list(G.nodes())
    n = len(nodes)
    for attempt in range(max_attempts):
        random.shuffle(nodes)
        A = set(nodes[:n//2])
        B = set(nodes[n//2:])
        improved = True
        while improved:
            improved = False
            for v in list(G.nodes()):
                if v in A:
                    d_int = sum(1 for u in G.neighbors(v) if u in A)
                    d_ext = sum(1 for u in G.neighbors(v) if u in B)
                    if d_ext > d_int and len(A) > 1:
                        A.remove(v); B.add(v)
                        improved = True; break
                else:
                    d_int = sum(1 for u in G.neighbors(v) if u in B)
                    d_ext = sum(1 for u in G.neighbors(v) if u in A)
                    if d_ext > d_int and len(B) > 1:
                        B.remove(v); A.add(v)
                        improved = True; break
            if is_satisfying(G, A, B):
                return A, B
    return A, B
```


Algorithme basé sur les cycles (extrait)

```
def satisfying_partition_from_cycles(G, C1, C2):
    A = set(C1)
    B = set(G.nodes()) - A
    locked = set(C1) | set(C2)  # ces sommets ne bougent pas
    improved = True
    while improved:
        improved = False
        for v in G.nodes():
            if v in locked:
                continue
            if v in A:
                d_ext = sum(1 for u in G.neighbors(v) if u in B)
                if d_ext >= 2:  # non satisfait (graphe 3- ou 4-regulier)
                    A.remove(v); B.add(v)
                    improved = True; break
            else:
                d_ext = sum(1 for u in G.neighbors(v) if u in A)
                if d_ext >= 2:
                    B.remove(v); A.add(v)
                    improved = True; break
    return A, B
```

Avantage

Garantit une partition satisfaisante quand elle existe, grâce au verrou sur C_1 et C_2 .

Ce que nous avons établi

- **Preuve constructive** de la conjecture pour $k \leq 4$.
- Formalisation rigoureuse via le concept de t -noyau.
- Preuve du lemme des deux cycles sommet-disjoints pour $k = 4$.
- Condition suffisante ($p < n/(\lceil k/2 \rceil + 1)$) pour l'existence d'un second noyau disjoint.
- Expérimentation algorithmique : aucun contre-exemple pour $k = 5$ parmi les petits graphes.

Saut de complexité

Pour $k \geq 5$, un simple cycle ne suffit plus comme ancrage. La structure des t -noyaux devient non-linéaire et bien plus difficile à caractériser.

❶ Généralisation de la recherche de noyaux

Borner l'ordre minimal n à partir duquel un premier t -noyau est garanti dans un graphe k -régulier.

❷ Optimisation algorithmique

Remplacer les heuristiques par un **solveur SAT** ou une formulation en **programmation linéaire en nombres entiers (PLNE)** pour étendre la vérification à des graphes de plus grande taille.

❸ Partitions équilibrées

Étudier les *Balanced Satisfactory Partitions* où $||A| - |B|| \leq 1$, avec des applications directes en partitionnement de réseaux de neurones.

Code source disponible sur https://github.com/Ivan23BG/TER_M1



P. Bärnkopf, Z. L. Nagy, Z. Paulovics.

A note on internal partitions: the 5-regular case and beyond.

Graphs and Combinatorics, 40(2):36, 2024.



C. Bazgan, Z. Tuza, D. Vanderpooten.

Satisfactory graph partition, variants, and generalizations.

European Journal of Operational Research, 206(2):271–280, 2010.



A. Gaikwad, S. Maity, S. K. Tripathi.

The satisfactory partition problem.

arXiv:2007.14339, 2020.



I. G. Yero, J. A. Rodríguez-Velázquez.

Defensive alliances in graphs: a survey.

arXiv:1308.2096, 2013.

Merci pour votre attention

Questions ?

Ivan LEJEUNE | Encadrant : Stéphane BESSY
Université de Montpellier — Master 1 Algorithmique — mai 2026